

```

/*****
* CHS ATHLETICS SCHEDULE SYSTEM
*
* Google Sheets
*   ↓
* Google Apps Script
*   ↓
*   |— Website API
*   |— Add Game
*   |— Edit Game
*   |— Delete Game
*   |— iCalendar (.ics) Export
*
* Google Sheets is the source of truth.
*****/

```

```

/* =====
SETTINGS
===== */

```

```

// CHS ATHLETICS GOOGLE SHEET
const SPREADSHEET_ID =
  '1fn5T06dE9nU2bJc7IRBydx2vnTbSpD0XrFWN0eut7vU';

```

```

// EXACT SHEET / TAB NAME
const SHEET_NAME =
  'CHS Athletics Schedule (SAMPLE DATA)';

```

```

/* =====
COLUMN NAMES

```

```

===== */

const HEADERS = [
  'Date',
  'Sport',
  'Team',
  'Level',
  'Opponent',
  'Time',
  'Location',
  'Home/Away',
  'Status',
  'Score',
  'Notes'
];

/* =====
MAIN WEB APP
===== */

function doGet(e) {

  try {

    e = e || {};

    const params =
      e.parameter || {};

    /* =====
ICS EXPORT

```

```

===== */

if (
  params.calendar === 'ics'
) {

  return createICSResponse();

}

/* =====
   ADD GAME
   ===== */

if (
  params.action === 'add'
) {

  if (!params.data) {

    throw new Error(
      'No game data was received.'
    );

  }

  const game =
    JSON.parse(
      params.data
    );

  const result =

```

```
        addGame(game);

    return sendResponse(
        e,
        result
    );

}

/* =====
   UPDATE GAME
   ===== */

if (
    params.action === 'update'
) {

    if (!params.data) {

        throw new Error(
            'No game data was received.'
        );

    }

    const game =
        JSON.parse(
            params.data
        );

    const result =
        updateGame(game);
```

```
    return sendResponse(  
        e,  
        result  
    );  
  
}  
  
/* =====  
   DELETE GAME  
   ===== */  
  
if (  
    params.action === 'delete'  
) {  
  
    if (!params.id) {  
  
        throw new Error(  
            'No game ID was received.'  
        );  
  
    }  
  
    const result =  
        deleteGame(  
            params.id  
        );  
  
    return sendResponse(  
        e,  
        result  
    );  
}
```

```

    );

}

/* =====
   NORMAL SCHEDULE API
   ===== */

const data =
  getScheduleData();

return sendResponse(
  e,
  {
    success: true,
    data: data
  }
);

} catch (error) {

  const errorResponse = {

    success: false,

    error:
      'Unable to process the athletics schedule request.',

    message:
      error.message

```

```

};

return sendResponse(
    e,
    errorResponse
);

}
}

/* =====
RESPONSE / JSONP
===== */

function sendResponse(
    e,
    response
) {

    const json =
        JSON.stringify(
            response
        );

    if (
        e &&
        e.parameter &&
        e.parameter.callback
    ) {

```

```
const callback =
    e.parameter.callback;

return ContentService

    .createTextOutput(
        callback +
        '(' +
        json +
        ');'
    )

    .setMimeType(
        ContentService.MimeType.JAVASCRIPT
    );
}

return ContentService

    .createTextOutput(
        json
    )

    .setMimeType(
        ContentService.MimeType.JSON
    );
}
```

```

/* =====
GET SCHEDULE SHEET
===== */

function getScheduleSheet() {

  const spreadsheet =
    SpreadsheetApp.openById(
      SPREADSHEET_ID
    );

  const sheet =
    spreadsheet.getSheetByName(
      SHEET_NAME
    );

  if (!sheet) {

    const availableSheets =
      spreadsheet
        .getSheets()
        .map(
          function(s) {

            return (
              s.getName() +
              ' (GID: ' +
              s.getSheetId() +
              ')'
            );
          }
        );
  }
}

```

```

        }
    )
    .join(', ');

    throw new Error(
        'Could not find sheet "' +
        SHEET_NAME +
        '". Available sheets: ' +
        availableSheets
    );
}

return sheet;
}

/* =====
GET SCHEDULE DATA
===== */

function getScheduleData() {

    const sheet =
        getScheduleSheet();

    const values =
        sheet
            .getDataRange()
            .getDisplayValues();

```

```

if (
    !values ||
    values.length === 0
) {

    return [];

}

/* =====
   HEADER ROW
   ===== */

const headers =
    values[0].map(
        function(header) {

            return clean(
                header
            );

        }
    );

/* =====
   FIND COLUMNS
   ===== */

const dateColumn =

```

```
findHeader(  
  headers,  
  [  
    'date',  
    'schedule date'  
  ]  
);
```

```
const sportColumn =  
  findHeader(  
    headers,  
    [  
      'sport'  
    ]  
  );
```

```
const teamColumn =  
  findHeader(  
    headers,  
    [  
      'team'  
    ]  
  );
```

```
const levelColumn =  
  findHeader(  
    headers,  
    [  
      'level'  
    ]  
  );
```

```
);
```

```
const opponentColumn =  
  findHeader(  
    headers,  
    [  
      'opponent'  
    ]  
  );
```

```
const timeColumn =  
  findHeader(  
    headers,  
    [  
      'time'  
    ]  
  );
```

```
const locationColumn =  
  findHeader(  
    headers,  
    [  
      'location'  
    ]  
  );
```

```
const homeAwayColumn =  
  findHeader(  
    headers,
```

```
[  
  'home/away',  
  'home away'  
]  
);
```

```
const statusColumn =  
  findHeader(  
    headers,  
    [  
      'status'  
    ]  
  );
```

```
const scoreColumn =  
  findHeader(  
    headers,  
    [  
      'score'  
    ]  
  );
```

```
const notesColumn =  
  findHeader(  
    headers,  
    [  
      'notes'  
    ]  
  );
```

```
/* =====  
   REQUIRED COLUMNS  
   ===== */  
  
const missingColumns = [];  
  
if (  
    dateColumn === -1  
) {  
  
    missingColumns.push(  
        'Date'  
    );  
  
}  
  
if (  
    sportColumn === -1  
) {  
  
    missingColumns.push(  
        'Sport'  
    );  
  
}  
  
if (  
    teamColumn === -1  
) {
```

```
missingColumns.push(  
    'Team'  
);  
  
}  
  
if (  
    opponentColumn === -1  
) {  
  
    missingColumns.push(  
        'Opponent'  
    );  
  
}  
  
if (  
    timeColumn === -1  
) {  
  
    missingColumns.push(  
        'Time'  
    );  
  
}  
  
if (  
    locationColumn === -1  
) {
```

```

        missingColumns.push(
            'Location'
        );
    }

    if (
        missingColumns.length > 0
    ) {

        throw new Error(
            'Missing required columns: ' +
            missingColumns.join(', ') +
            '. Your header row is: ' +
            values[0].join(' | ')
        );
    }

    /* =====
    BUILD GAME OBJECTS
    ===== */

    const games = [];

    for (
        let row = 1;
        row < values.length;
        row++

```

```

) {

    const currentRow =
        values[row];

    /* -----
       Skip blank rows
       ----- */

    const hasData =
        currentRow.some(
            function(cell) {

                return (
                    clean(cell) !== ''
                );

            }
        );

    if (!hasData) {
        continue;
    }

    /* -----
       Date
       ----- */

    const scheduleDate =
        getCell(

```

```

        currentRow,
        dateColumn
    );

    /*
     * Ignore rows without a date.
     */

    if (!scheduleDate) {
        continue;
    }

    /* -----
     * GAME OBJECT
     * ----- */

    const game = {

        /*
         * Google Sheet row number.
         *
         * Header is row 1,
         * therefore first game is row 2.
         */

        id:
            String(row + 1),

        scheduleDate:
            scheduleDate,

```

```
sport:
    getCell(
        currentRow,
        sportColumn
    ),
```

```
team:
    getCell(
        currentRow,
        teamColumn
    ),
```

```
level:
    getCell(
        currentRow,
        levelColumn
    ),
```

```
opponent:
    getCell(
        currentRow,
        opponentColumn
    ),
```

```
time:
    getCell(
        currentRow,
```

```
        timeColumn
    ),

    location:
        getCell(
            currentRow,
            locationColumn
        ),

    homeAway:
        getCell(
            currentRow,
            homeAwayColumn
        ),

    status:
        getCell(
            currentRow,
            statusColumn
        ),

    score:
        getCell(
            currentRow,
            scoreColumn
        ),

    notes:
```

```

        getCell(
            currentRow,
            notesColumn
        )

    };

    games.push(
        game
    );

}

return games;
}

/* =====
ADD GAME
===== */

function addGame(
    game
) {

    validateGame(
        game
    );

    const sheet =

```

```
getScheduleSheet();

/*
 * Write in the exact same
 * order as the Google Sheet.
 */

const row = [

  game.scheduleDate || '',

  game.sport || '',

  game.team || '',

  game.level || '',

  game.opponent || '',

  game.time || '',

  game.location || '',

  game.homeAway || '',

  game.status || 'Scheduled',

  game.score || '',

  game.notes || ''

];
```

```
sheet.appendRow(  
    row  
);
```

```
SpreadsheetApp.flush();
```

```
return {  
  
    success: true,  
  
    message:  
        'Game added successfully.'  
  
};  
}
```

```
/* =====  
    UPDATE GAME  
===== */
```

```
function updateGame(  
    game  
) {  
  
    validateGame(  
        game  
    );  
}
```

```
if (
  !game.id
) {

  throw new Error(
    'Game ID is required.'
  );

}

const sheet =
  getScheduleSheet();

const rowNumber =
  Number(
    game.id
  );

/*
 * Make sure this is
 * a valid game row.
 */

if (
  !Number.isInteger(
    rowNumber
  ) ||
  rowNumber < 2 ||
  rowNumber > sheet.getLastRow()
```

```
) {  
  
  throw new Error(  
    'The selected game could not be found.'  
  );  
  
}
```

```
const row = [  
  
  game.scheduleDate || '',  
  
  game.sport || '',  
  
  game.team || '',  
  
  game.level || '',  
  
  game.opponent || '',  
  
  game.time || '',  
  
  game.location || '',  
  
  game.homeAway || '',  
  
  game.status || 'Scheduled',  
  
  game.score || '',  
  
  game.notes || ''
```

```
];
```

```
sheet
```

```
  .getRange(  
    rowNumber,  
    1,  
    1,  
    HEADERS.length
```

```
  )
```

```
  .setValues([
```

```
    row
```

```
  ]);
```

```
SpreadsheetApp.flush();
```

```
return {
```

```
  success: true,
```

```
  message:
```

```
    'Game updated successfully.'
```

```
};
```

```
}
```

```
/* =====
```

```
DELETE GAME
```

```
===== */
```

```
function deleteGame(  
  id  
) {  
  
  if (!id) {  
  
    throw new Error(  
      'Game ID is required.'  
    );  
  
  }  
  
  const sheet =  
    getScheduleSheet();  
  
  const rowNumber =  
    Number(  
      id  
    );  
  
  if (  
    !Number.isInteger(  
      rowNumber  
    ) ||  
    rowNumber < 2 ||  
    rowNumber > sheet.getLastRow()  
  ) {  
  
    throw new Error(  
      'The selected game could not be found.'  
    );  
  }  
}
```

```

    );

}

sheet.deleteRow(
    rowNumber
);

SpreadsheetApp.flush();

return {

    success: true,

    message:
        'Game deleted successfully.'

};
}

/* =====
VALIDATE GAME
===== */

function validateGame(
    game
) {

    if (!game) {

```

```
    throw new Error(  
        'Game information is missing.'  
    );  
  
}
```

```
if (  
    !game.scheduleDate  
) {  
  
    throw new Error(  
        'Please enter a date.'  
    );  
  
}
```

```
if (  
    !game.sport  
) {  
  
    throw new Error(  
        'Please select a sport.'  
    );  
  
}
```

```
if (  
    !game.team  
) {
```

```
    throw new Error(  
        'Please enter the team.'  
    );  
  
}  
  
if (  
    !game.level  
) {  
  
    throw new Error(  
        'Please select the level.'  
    );  
  
}  
  
if (  
    !game.opponent  
) {  
  
    throw new Error(  
        'Please enter the opponent.'  
    );  
  
}  
  
if (  
    !game.time  
) {
```

```
    throw new Error(  
        'Please enter the game time.'  
    );  
  
}
```

```
if (  
    !game.location  
) {
```

```
    throw new Error(  
        'Please enter the location.'  
    );  
  
}
```

```
if (  
    !game.homeAway  
) {
```

```
    throw new Error(  
        'Please select Home or Away.'  
    );  
  
}
```

```
if (  
    !game.status  
) {
```

```

    throw new Error(
        'Please select a status.'
    );

}

}

/* =====
FIND HEADER
===== */

function findHeader(
    headers,
    possibleNames
) {

    for (
        let i = 0;
        i < headers.length;
        i++
    ) {

        const header =
            clean(
                headers[i]
            );

        for (
            let j = 0;
            j < possibleNames.length;

```

```

        j++
    ) {

        if (
            header ===
            clean(
                possibleNames[j]
            )
        ) {

            return i;

        }

    }

}

return -1;
}

/* =====
   GET CELL
   ===== */

function getCell(
    row,
    columnIndex
) {

    if (

```

```

        columnIndex === -1 ||
        columnIndex === undefined ||
        columnIndex >= row.length
    ) {

        return '';

    }

    return String(
        row[columnIndex] || ''
    ).trim();
}

/* =====
   CLEAN
   ===== */

function clean(
    value
) {

    return String(
        value || ''
    )
        .trim()
        .toLowerCase()
        .replace(
            /\s+/g,
            ' '
        );
}

```

```
}

/* =====
TEST CONNECTION
===== */

function testScheduleConnection() {

  const data =
    getScheduleData();

  Logger.log(
    'Athletics games loaded: ' +
    data.length
  );

  if (
    data.length > 0
  ) {

    Logger.log(
      JSON.stringify(
        data[0],
        null,
        2
      )
    );
  }
}
```

```

    return data;
}

/* =====
ICS RESPONSE
===== */

function createICSResponse() {

    const games =
        getScheduleData();

    const ics =
        buildICS(
            games
        );

    return ContentService
        .createTextOutput(
            ics
        )
        .setMimeType(
            ContentService.MimeType.TEXT
        );
}

/* =====
BUILD ICS
===== */

```

```
===== */
```

```
function buildICS(
  games
) {

  const lines = [];

  lines.push(
    'BEGIN:VCALENDAR'
  );

  lines.push(
    'VERSION:2.0'
  );

  lines.push(
    'PRODID:-//CHS Athletics//Athletics Schedule//EN'
  );

  lines.push(
    'CALSCALE:GREGORIAN'
  );

  lines.push(
    'METHOD:PUBLISH'
  );

  lines.push(
    'X-WR-CALNAME:CHS Athletics Schedule'
  );
}
```

```
games.forEach(  
  function(game, index) {  
  
    const start =  
      getICSStart(  
        game  
      );  
  
    if (!start) {  
      return;  
    }  
  
    const end =  
      getICSEnd(  
        start  
      );  
  
    lines.push(  
      'BEGIN:VEVENT'  
    );  
  
    lines.push(  
      'UID:' +  
      escapeICS(  
        createICSUID(  
          game,  
          index  
        )  
      )  
    );  
  }  
);
```

```
)  
);  
  
lines.push(  
    'DTSTAMP:' +  
    getUTCStamp()  
);  
  
lines.push(  
    'DTSTART:' +  
    start  
);  
  
lines.push(  
    'DTEND:' +  
    end  
);  
  
lines.push(  
    'SUMMARY:' +  
    escapeICS(  
        buildICSTitle(  
            game  
        )  
    )  
);  
  
if (
```

```
        game.location
    ) {

        lines.push(
            'LOCATION:' +
            escapeICS(
                game.location
            )
        );

    }

    lines.push(
        'DESCRIPTION:' +
        escapeICS(
            buildICSDescription(
                game
            )
        )
    );

    lines.push(
        'END:VEVENT'
    );

}

);

lines.push(
    'END:VCALENDAR'
```

```

);

return (
    lines.join(
        '\r\n'
    ) +
    '\r\n'
);
}

/* =====
ICS TITLE
===== */

function buildICSTitle(
    game
) {

    let title =
        'CHS Athletics';

    if (
        game.sport
    ) {

        title =
            game.sport;

    }
}

```

```
if (  
  game.team  
) {  
  
  title +=  
    ' - ' +  
    game.team;  
  
}
```

```
if (  
  game.opponent  
) {  
  
  if (  
    game.homeAway === 'Home'  
  ) {  
  
    title +=  
      ' vs. ' +  
      game.opponent;  
  
  } else {  
  
    title +=  
      ' @ ' +  
      game.opponent;  
  
  }  
  
}
```

```
    return title;
}

/* =====
   ICS DESCRIPTION
   ===== */

function buildICSDescription(
    game
) {

    const parts = [];

    if (
        game.team
    ) {

        parts.push(
            'Team: ' +
            game.team
        );

    }

    if (
        game.level
    ) {
```

```
    parts.push(  
      'Level: ' +  
      game.level  
    );  
  
  }  
  
  if (  
    game.opponent  
  ) {  
  
    parts.push(  
      'Opponent: ' +  
      game.opponent  
    );  
  
  }  
  
  if (  
    game.homeAway  
  ) {  
  
    parts.push(  
      'Home/Away: ' +  
      game.homeAway  
    );  
  
  }  
  
  if (  
    game.opponent
```

```
    game.status  
  ) {  
  
    parts.push(  
      'Status: ' +  
      game.status  
    );  
  
  }
```

```
  if (  
    game.score  
  ) {  
  
    parts.push(  
      'Score: ' +  
      game.score  
    );  
  
  }
```

```
  if (  
    game.notes  
  ) {  
  
    parts.push(  
      'Notes: ' +  
      game.notes  
    );  
  
  }
```

```
parts.push(
  'Source: CHS Athletics Schedule'
);

return parts.join(
  '\n'
);
}

/* =====
ICS START
===== */

function getICSStart(
  game
) {

  const date =
    convertDateToISO(
      game.scheduleDate
    );

  if (!date) {
    return '';
  }

  const time =
```

```
    parseTime(  
        game.time  
    );  
  
    /*  
    * No time = all-day event.  
    */  
  
    if (!time) {  
  
        return date.replace(  
            /-/g,  
            ''  
        );  
  
    }  
  
    return (  
        date.replace(  
            /-/g,  
            ''  
        ) +  
        'T' +  
        String(  
            time.hour  
        ).padStart(  
            2,  
            '0'  
        ) +  
        String(  
            time.minute
```

```

        ).padStart(
            2,
            '0'
        ) +
        '00'
    );
}

/* =====
ICS END
===== */

function getICSEnd(
    start
) {

    /*
     * All-day event.
     */

    if (
        start.length === 8
    ) {

        const year =
            Number(
                start.substring(
                    0,
                    4
                )
            );
    }
}

```

```
const month =  
  Number(  
    start.substring(  
      4,  
      6  
    )  
  ) - 1;
```

```
const day =  
  Number(  
    start.substring(  
      6,  
      8  
    )  
  );
```

```
const date =  
  new Date(  
    year,  
    month,  
    day + 1  
  );
```

```
return (  
  date.getFullYear() +  
  String(  
    date.getMonth() + 1  
  ).padStart(  
    2,
```

```
        '0'
    ) +
    String(
        date.getDate()
    ).padStart(
        2,
        '0'
    )
);

}

/*
 * Default game duration:
 * 2 hours.
 */

const year =
    Number(
        start.substring(
            0,
            4
        )
    );

const month =
    Number(
        start.substring(
            4,
            6
        )
    )
```

```
) - 1;
```

```
const day =  
  Number(  
    start.substring(  
      6,  
      8  
    )  
  );
```

```
const hour =  
  Number(  
    start.substring(  
      9,  
      11  
    )  
  );
```

```
const minute =  
  Number(  
    start.substring(  
      11,  
      13  
    )  
  );
```

```
const date =  
  new Date(  
    year,
```

```
    month,  
    day,  
    hour,  
    minute  
);
```

```
date.setHours(  
    date.getHours() + 2  
);
```

```
return (  
    date.getFullYear() +  
    String(  
        date.getMonth() + 1  
    ).padStart(  
        2,  
        '0'  
    ) +  
    String(  
        date.getDate()  
    ).padStart(  
        2,  
        '0'  
    ) +  
    'T' +  
    String(  
        date.getHours()  
    ).padStart(  
        2,  
        '0'  
    ) +
```

```

    String(
        date.getMinutes()
    ).padStart(
        2,
        '0'
    ) +
        '00'
);
}

/* =====
PARSE TIME
===== */

function parseTime(
    value
) {

    const text =
        String(
            value || ''
        )
        .trim()
        .toLowerCase();

    const match =
        text.match(
            /(\d{1,2})(?::(\d{2}))?\s*(am|pm)?/
        );

```

```
if (!match) {  
    return null;  
}
```

```
let hour =  
    Number(  
        match[1]  
    );
```

```
const minute =  
    Number(  
        match[2] || 0  
    );
```

```
const period =  
    match[3];
```

```
if (  
    period === 'pm' &&  
    hour !== 12  
) {
```

```
    hour += 12;
```

```
}
```

```
if (  
    period === 'am' &&
```

```

        hour === 12
    ) {

        hour = 0;

    }

    return {

        hour:
            hour,

        minute:
            minute

    };
}

/* =====
   CONVERT DATE TO ISO
   ===== */

function convertDateToISO(
    value
) {

    const text =
        String(
            value || ''
        ).trim();

```

```
if (!text) {  
    return '';  
}
```

```
/*  
 * YYYY-MM-DD  
 */
```

```
if (  
    /^\\d{4}-\\d{2}-\\d{2}$/.test(  
        text  
    )  
) {  
  
    return text;  
  
}
```

```
/*  
 * MM/DD/YYYY  
 */
```

```
const match =  
    text.match(  
        /^(\\d{1,2})\\/(\\d{1,2})\\/(\\d{4})$/  
    );
```

```
if (match) {
```

```

return (
    match[3] +
    '-' +
    String(
        match[1]
    ).padStart(
        2,
        '0'
    ) +
    '-' +
    String(
        match[2]
    ).padStart(
        2,
        '0'
    )
);

}

return '';
}

/* =====
ICS UID
===== */

function createICSUID(
    game,
    index
) {

```

```

const text =
[
    game.id,
    game.scheduleDate,
    game.sport,
    game.team,
    game.opponent,
    game.time,
    index
]
    .join(
        ' _ '
    )
    .replace(
        /[^\a-zA-Z0-9-]/g,
        ''
    );

return (
    text +
    '@chs-athletics'
);
}

/* =====
   UTC TIMESTAMP
   ===== */

function getUTCStamp() {

```

```
const now =  
  new Date();  
  
return (  
  now.getUTCFullYear() +  
  String(  
    now.getUTCMonth() + 1  
  ).padStart(  
    2,  
    '0'  
  ) +  
  String(  
    now.getUTCDate()  
  ).padStart(  
    2,  
    '0'  
  ) +  
  'T' +  
  String(  
    now.getUTCHours()  
  ).padStart(  
    2,  
    '0'  
  ) +  
  String(  
    now.getUTCMinutes()  
  ).padStart(  
    2,  
    '0'  
  ) +  
  String(  
    now.getUTCSeconds()  
  )
```

```

        ).padStart(
            2,
            '0'
        ) +
        'Z'
    );
}

/* =====
   ESCAPE ICS
   ===== */

function escapeICS(
    value
) {

    return String(
        value || ''
    )
        .replace(
            /\//g,
            '\\\\'
        )
        .replace(
            /;/g,
            '\\;'
        )
        .replace(
            /,/g,
            '\\,'
        )
        .replace(

```

```
    /\r?\n/g,  
    '\\n'  
  );  
}
```